Technical Report

Edge-Based Anomaly Detection For Fleet Based Condition Monitoring of Machines

Submitted By

Dwarakesh Venkatesh, CB.SC.U4CSE23020

Shyam Sundar Raju, CB.SC.U4CSE23049

Sarigasini M, CB.SC.U4CSE23065

Pooja Shree S, CB.SC.U4CSE23346

in partial fulfilment of the requirements for the course of

23CSE362 - EDGE COMPUTING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112

List of Tables

List of Figures

List of Abbreviations

MQTT Message Queuing Telemetry Transport

M2M Machine to Machine

UI User Interface

Contents

1 Abstract

Industrial machinery requires maximum reliability, as unexpected breakdowns can lead to production losses, high maintenance costs, and safety hazards. Traditional condition monitoring systems rely heavily on handcrafted rules or supervised models, both of which require extensive expert involvement and large labeled datasets. However, existing approaches face significant challenges including high latency due to centralized cloud processing, bandwidth constraints, and limited real-time adaptability in dynamic industrial environments. This work implements an edge-based anomaly detection framework integrated within a three-tier edge–fog–cloud architecture that enables distributed, real-time fault detection for machine fleets. The edge layer performs local preprocessing and anomaly detection, the fog layer monitors edge-node health via MQTT heartbeats, and the cloud layer provides fleet-wide visualization through a responsive dashboard. The framework employs unsupervised hierarchical agglomerative clustering to identify anomalies based on inter-machine similarity without requiring labeled failure data, with anomaly scores computed as the fraction of machines outside each cluster. Performance evaluation demonstrates that accuracy improves from 93% to 97% as fleet size increases from 50 to 300 machines, while maintaining acceptable resource utilization (40% CPU, 140 MB RAM at maximum fleet size). The distributed architecture achieves low end-to-end latency (14.4 ms for 100 machines at 100 Mbps), scalable fleet management, and interpretable anomaly visualization, making it well-suited for industrial IoT environments with limited connectivity.

Keywords: Anomaly Detection, Agglomerative Clustering, Fleet Monitoring, Edge Computing, MQTT, Distributed Architecture

2 Introduction

2.1 Background

Machine fault detection is of utmost importance in industrial environments where machines are expected to operate with maximum reliability, leaving no room for failure. When breakdowns occur, it results in technical faults, production losses and unexpected downtime, substantial repair and maintenance costs, serious injuries or loss of life, and potential environmental damage. The manufacturing and process industries have increasingly recognized that traditional reactive maintenance approaches are insufficient to meet the demands of modern industrial operations, where even brief periods of unplanned downtime can result in ripple effects throughout the production chain [1].

Recent advances in sensor technologies and the Industrial Internet of Things (IIoT) have enabled continuous monitoring of machine conditions through various physical parameters such as vibration, temperature, current, and acoustic signatures. However, the challenge lies not only in collecting this data but in effectively analyzing it to detect anomalies that may indicate impending failures. The complexity of modern industrial machinery, combined with varying operational conditions and environmental factors, makes this task particularly challenging [6].

2.2 Drawbacks of Existing Solutions

Traditional machine fault detection methods have relied heavily on manual threshold setting and rule-based systems, where domain experts define specific indicators and thresholds for each type of machine and fault condition. This handcrafted approach is not only time-consuming, requiring weeks of expert analysis to calibrate appropriate thresholds for each motor type and configuration, but also lacks scalability. Each new machine or operational setup requires its own set of custom rules that must account for varying working conditions including location, humidity, and temperature variations [3].

Supervised learning approaches have emerged as an alternative, leveraging historical data to train predictive models. However, these methods face significant practical limitations in industrial settings. They require extensive labeled datasets containing thousands of examples for each fault type, which are often difficult and expensive to collect. [1]. Furthermore, supervised models demand continuous human involvement for labeling of historical logs, validation of predictions, and regular adjustment of model parameters as operating conditions evolve. The data diversity requirement is particularly challenging, as models

must be trained on examples that represent every possible fault type across different operational scenarios.

Semi-supervised learning methods attempt to reduce the labeling need by utilizing both labeled and unlabeled data. Nevertheless, these approaches often ignore critical contextual factors such as environmental conditions, leading to misclassification of normal operational variations as faults. For instance, a model might incorrectly flag elevated temperatures resulting from seasonal heat variations as machine anomalies [4]. Moreover, semi-supervised methods exhibit limited real-time adaptability, requiring complete retraining rather than simple reconfiguration when there is sudden changes in workload or operational patterns.

2.3 Motivation

Despite significant advancements in condition monitoring and predictive maintenance, most existing machine fault detection systems remain heavily dependent on cloud-based analysis or supervised learning models. These approaches encounter several fundamental limitations that hinder their practical deployment in real-world industrial environments. Cloud-centric architectures introduce latency due to data transmission and centralized processing, require substantial communication bandwidth, and create vulnerabilities in network connectivity. In industrial settings where machines operate continuously and often in remote or harsh environments, these constraints render cloud-only approaches impractical [2].

Moreover, traditional models frequently fail to account for the dynamic and context-dependent nature of industrial operations. Machines do not operate in isolation but function within complex systems where operational and environmental conditions vary continuously. A temperature anomaly might simply reflect seasonal variation or increased ambient conditions rather than indicating an actual fault. Such scenarios underscore the critical need for intelligent decision-making systems that can distinguish between normal operational variations and genuine anomalies while operating closer to the data source [7].

Edge computing presents a promising paradigm shift that enables computation and analysis at or near the point of data generation. By processing sensor data locally at edge nodes, faults can be detected in real-time without constant dependency on cloud connectivity. This approach not only reduces latency and bandwidth consumption but also enhances data privacy, system resilience, and enables autonomous operation even when network connectivity is compromised [8]. The integration of unsupervised learning methods with edge computing architectures offers particular promise for industrial anomaly detection, as these

methods do not require extensive labeled datasets and can adapt to normal operational variations through continuous learning [10].

2.4 Problem Statement

Industrial condition monitoring needs *real-time*, low-latency fault detection that remains reliable under bandwidth limits and intermittent connectivity. Cloud-centric pipelines incur latency and communication overhead [2], while supervised models depend on large labeled datasets that are costly and impractical when faults are rare [1, 5]. An edge-based, *unsupervised* approach is therefore preferred, leveraging the fact that most machines operate normally and deviations are indicative of faults [1, 6].

3 Literature Survey

The rapid advancement of Industrial Internet of Things (IIoT) technologies has driven the need for anomaly detection systems capable of monitoring machine health in real-time while addressing challenges related to scalability, data availability, and computational constraints. This section reviews the current state-of-the-art approaches in machine fault detection, focusing on fleet-based monitoring, unsupervised learning methodologies, and distributed computing paradigms.

3.1 Fleet-Based Monitoring and Unsupervised Learning Approaches

The performance and reliability of machines are critical for several industries, as failures or malfunctions may lead to high production losses, severe injuries, or even loss of lives. Hendrickx et al. [1] propose an unsupervised, generic anomaly detection framework specifically designed for fleet-based condition monitoring that eliminates the requirement for historical labeled datasets by performing online fleet-based comparisons. Their framework offers three key advantages: the absence of historical data requirements, the ability to incorporate domain expertise through user-defined comparison measures, and easy interpretability that enables domain experts to validate predictions. The framework demonstrates its applicability through two use-cases on electrical machine fleets for detecting voltage unbalances using electrical and vibration signatures.

Truong et al. [2] propose a lightweight federated learning-based architecture for anomaly detection in time-series data within industrial control systems. Their hybrid design incorporating federated learning, autoencoders, transformers, and Fourier mixing sublayers achieves high accuracy while maintaining computational efficiency suitable for deployment on edge devices with limited computing capacity. Wong et al. [3] contribute a modified self-organizing map for automated novelty detection applied to vibration signal monitoring, combining Kohonen's self-organizing map with a multidimensional dissimilarity measure for dual-class classification. Their modular approach demonstrates high accuracy and robustness across different condition monitoring applications using real-world vibration data from multi-sensor environments with up to eight sensors.

3.2 Domain-Specific Methodologies and Algorithm Comparisons

Specialized methodologies tailored to specific industrial contexts have emerged to address unique challenges in different operational environments. Derse et al. [4] present a comprehensive study on anomaly detection for automotive sensor data time series, comparing the performance of various algorithms including

interquartile range, isolation forest, particle swarm optimization, and k-means clustering. Their work addresses the multi-source big data problem arising from numerous sensors in modern vehicles that generate over a gigabyte of data per second, highlighting the importance of selecting appropriate algorithms based on specific characteristics of automotive sensor data and operational requirements.

Rajagopal et al. [5] conduct a comparative analysis of machine learning algorithms for anomaly detection, evaluating Isolation Forest against Random Forest, Decision Tree, K-means, and Logistic Regression using comprehensive metrics such as accuracy, precision, recall, F1 score, training time, and testing time. Their evaluation provides valuable insights into the trade-offs between model complexity, computational efficiency, and detection accuracy, offering practical guidance for practitioners selecting algorithms for specific industrial applications. Carratù et al. [6] propose a novel unsupervised methodology for anomaly detection in industrial electrical systems that analyzes electrical current values and other power grid parameters. Their framework combines both machine learning algorithms and traditional analysis techniques optimized for improved performance and execution time, including short-time Fourier transform analysis to capture temporal dynamics of anomalies. The methodology attains excellent performance with zero false positives and less than four percent undetected outlier events across all tested datasets.

3.3 Security-Focused and Distributed IoT Applications

As industrial IoT networks expand to encompass thousands of distributed sensors and edge devices, security-focused and privacy-preserving distributed learning paradigms have gained prominence. Zou et al. [7] provide a valuable case study demonstrating anomaly detection implementation in real industrial environments, focusing on Industrial Control Security. Their work illustrates the attacking process of virus spread and worm propagation in industrial settings while demonstrating how anomaly detection techniques can identify malicious behaviors and assist security administrators in enhancing system security.

Radhakrishnan et al. [8] propose an unsupervised representation learning approach for intrusion detection in Industrial Internet of Things network environments using Fuzzy C-Means algorithms with autoencoders. Their methodology achieves a maximum accuracy of ninety-seven percent without requiring labeled data, demonstrating superior performance compared to existing supervised frameworks. The work emphasizes that unsupervised methods do not expect data to be labeled, which is more practical since systems cannot be expected to know features that would cause attacks beforehand. Sivakumar et al. [9] extend the application of hybrid approaches by presenting an optimized ar-

chitecture combining AutoEncoder and TabNet models for anomaly detection of DDoS and network attacks in Internet of Medical Things systems. Their approach addresses the growing complexity of distributed denial of service assaults while considering the limited computational resources and diverse communication protocols characteristic of IoMT environments, achieving weighted F1-scores of one point zero in certain attack scenarios. Abhiram et al. [10] demonstrate the integration of anomaly detection capabilities with IoT platforms for real-time monitoring through an environmental monitoring system using ESP8266 microcontroller and DHT11 sensor. Their system provides real-time monitoring of temperature and humidity while generating alerts for threshold violations and sensor integrity issues through Arduino IoT Cloud connectivity, showcasing context awareness and adaptive monitoring features applicable to smart homes, farming, and industrial applications.

3.4 Challenges

Although significant progress has been made in machine fault detection, several practical challenges still hinder large-scale industrial adoption:

1. **Data Labeling:** Supervised models require large, labeled fault datasets, but labeling anomalies is costly and rare in real operations [1].

2. **System Diversity:** Machines within the same fleet often differ in design, operating range, and sensor configuration, making it difficult to create a single standardized model [3, 2].

3. **Low-Latency Processing:** Real-time detection must run efficiently on edge devices with limited compute and power, without relying on constant cloud connectivity [4, 5].

4. **Scalability:** Extending monitoring from a few machines to hundreds of distributed sensors increases communication load and system complexity [6].

5. **Privacy and Security:** Centralized cloud learning exposes sensitive operational data to network risks [7].

6. **Non-Stationary Data:** Machine behavior evolves due to wear, load, and environmental factors, causing static models to degrade over time [8].

7. **False Alarms:** Many systems fail to differentiate real faults from normal variations, such as temperature or workload changes [9, 10].

8. **Interpretability:** Maintenance teams need transparent, explainable results to trust and act on anomaly alerts.

Addressing these challenges requires a lightweight, adaptive, and interpretable framework that supports distributed, real-time fault detection across heterogeneous industrial fleets.

3.5 Research Gap

Table 1: Research Gaps in Fleet-Based Anomaly Detection and Industrial Condition Monitoring

Research Gap	**Description**	**References**
Integrated Edge-Fog-Cloud Architecture	Limited frameworks exist that comprehensively integrate edge processing, fog-layer aggregation, and cloud-based analytics for fleet-level anomaly detection while maintaining real-time responsiveness and minimizing communication overhead.	[1, 2, 10]
Context-Aware Fleet Monitoring	Existing fleet-based approaches lack sophisticated mechanisms to distinguish between genuine machine faults and benign operational variations caused by environmental conditions, seasonal changes, or workload fluctuations.	[3, 4, 6]
Lightweight Unsupervised Methods for Edge Devices	While unsupervised methods eliminate labeled data requirements, few approaches are optimized for deployment on resource-constrained edge devices typical in industrial IoT environments, balancing detection accuracy with computational efficiency.	[5, 8, 10]
Privacy-Preserving Distributed Learning	Integration of federated learning with real-time fleet-based anomaly detection remains nascent, particularly for industrial control systems where data privacy and network bandwidth constraints are critical concerns.	[7, 9]
Interpretable Anomaly Detection	Gap in developing interpretable anomaly detection frameworks that provide actionable insights to domain experts and maintenance personnel, enabling validation of predictions and informed decision-making for intervention strategies.	[1, 6, 9]

These research gaps collectively underscore the necessity for an integrated framework that synergistically combines fleet-level learning, unsupervised detection methodologies, distributed edge-fog-cloud architecture, context-aware analysis, and practical interpretability to address the multifaceted challenges of industrial condition monitoring in real-world deployment scenarios.

4 Problem Formulation

4.1 System Overview

In industrial environments, unexpected machine failures can result in significant downtime, costly repairs, and even safety hazards. Traditional condition monitoring methods depend heavily on handcrafted fault indicators and extensive labeled historical datasets. However, obtaining such data is often impractical because machines operate under numerous varying conditions, and potential faults are rare or unknown beforehand.

Furthermore, most existing artificial intelligence–based monitoring systems analyze only single machines and function as black-box models, offering limited interpretability to domain experts.

The problem, therefore, is to develop a general and interpretable anomaly detection framework capable of identifying faulty machines in a fleet without requiring labeled or historical data. The framework must be able to:

1. Continuously monitor multiple similar machines (a fleet) operating under comparable conditions.
2. Detect deviations in machine behavior that may indicate potential faults, assuming that most machines are healthy.
3. Operate in an unsupervised manner, eliminating the dependency on prelabeled datasets.
4. Provide interpretable results and visualizations that allow domain experts to validate and understand anomaly predictions.

Machines with anomaly scores above a predefined threshold are flagged as potentially faulty. The proposed framework addresses this problem through four interconnected modules: **machine comparison**, **fleet clustering**, **anomaly detection**, and **visualization**, enabling a scalable, interpretable, and data-efficient solution for real-time fleet condition monitoring.

5 Proposed Architecture

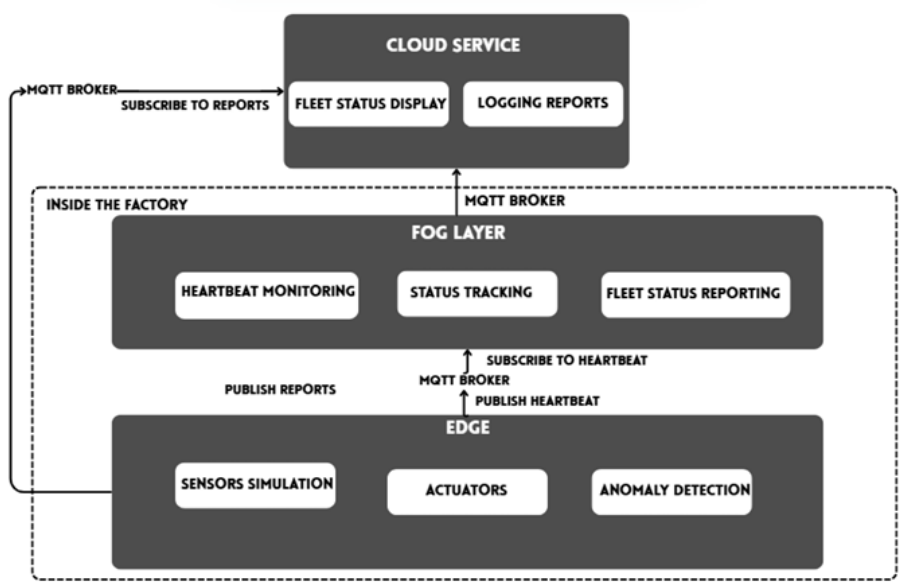


Figure 1: Overview of the Proposed Architecture

Step 1: Machine Comparison
Step 2: Fleet Clustering
Step 3: Anomaly Detection
Step 4: Visualization

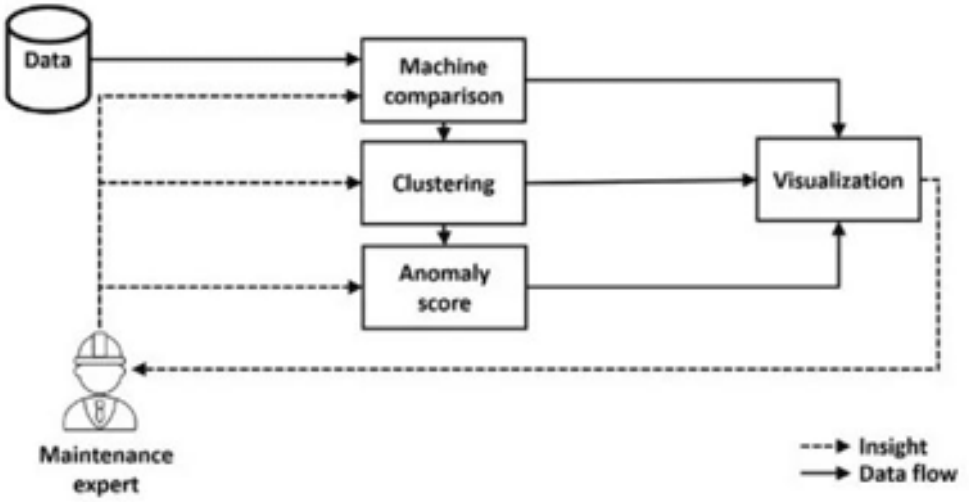


Figure 2: Layered System Design (Edge, Fog, and Cloud Layers)

5.1 Edge Layer

- **Sensor Simulation:** Generates realistic, time-varying data for multiple metrics like temperature, humidity, and pressure.
- **Local Data Buffering:** Maintains a rolling window (local buffer) of recent sensor readings.
- **Edge-based Anomaly Detection:** Performs real-time anomaly detection locally, reducing the need to send all raw data to the cloud.
- **MQTT Communication:** Uses the Message Queuing Telemetry Transport (MQTT) protocol for machine-to-machine (M2M) communication.
- **Publish Reports:** Sends detailed analysis reports (decision, anomaly score, metrics) to the cloud via fog for further processing.
- **Publish Heartbeats:** Sends regular "alive" status messages to the fog layer.
- **Actuator Triggering:** Can trigger actuators autonomously without instruction from cloud or fog.

5.2 Fog Layer

A heartbeat is a small, periodic signal that an edge node sends to a central controller (or monitoring system) to indicate that it is still active and functioning.

- **Heartbeat Monitoring:** Subscribes to a wildcard MQTT topic (`edge/+/heartbeat`) to listen for "alive" signals from all edge nodes. The payload might simply be the string "alive".
- **Status Tracking:** Maintains the status of each edge node. If a node fails to send a heartbeat within a configurable `HEARTBEAT_TIMEOUT`, it is marked as "unresponsive." When a heartbeat resumes, it is marked as "alive" again.
- **Fleet Status Reporting:** Periodically compiles a "fleet snapshot" summarizing the status of every edge node and publishes it to the cloud.

This mechanism ensures that edge devices remain functional and faults are detected without delay.

5.3 Cloud Layer

- **MQTT Subscriber:** Connects to an MQTT broker and subscribes to:
  - **Reports:** To receive anomaly analysis reports from each fog node.
  - **Fleet_Snapshot:** To receive periodic health summaries for the entire fleet.
- **Responsive UI:** The dashboard automatically refreshes once new data is received, providing a live system view at periodic intervals.
- **Fleet Snapshot:** Displays the overall fleet size and status, including a table and bar chart visualizing anomaly scores for all active nodes.
- **Live Report Stream:** Shows a continuously updating table of detailed reports as they arrive from edge nodes.
- **Failsafe anomaly detection:** In case if any edge node fails, the cloud takes control and uses the data received to run the anomaly detection algorithm until the edge node comes back online.

6 Methodology

6.1 Simulation Environment

The proposed architecture was implemented and analysed in a simulated environment using Docker containers to emulate a edge–fog–cloud system. The simulation was carried out on an Intel Core i7 processor with 16 GB of RAM laptop, serving as the host system for all containers. Each layer of the architecture—edge, fog, and cloud—was instantiated as an independent container with distinct configurations to reflect their real-world placement and computational capacities.

To simulate the constrained hardware conditions found in edge devices, each edge node container was assigned a resource limit of 124 MB of RAM and 0.5 virtual CPU cores. These constraints ensure that the anomaly detection model operates under realistic performance and memory pressure, making the evaluation closely aligned with industrial deployment scenarios. Despite the limited resources, the edge nodes successfully performed local preprocessing, clustering, and anomaly detection in real time.

On the other hand, the fog and cloud layers were configured without any explicit computational limits, representing high-performance servers in a production environment. The fog layer primarily handled heartbeat aggregation, fleet health monitoring, and data forwarding to the cloud, while the cloud layer managed large-scale data visualization, long-term analytics, and user interface updates.

This layered Docker-based simulation allowed for efficient testing of inter-layer communication, scalability with varying fleet sizes, and system behavior under constrained resources, providing valuable insights into the practicality of deploying edge-based anomaly detection in real-world industrial systems.

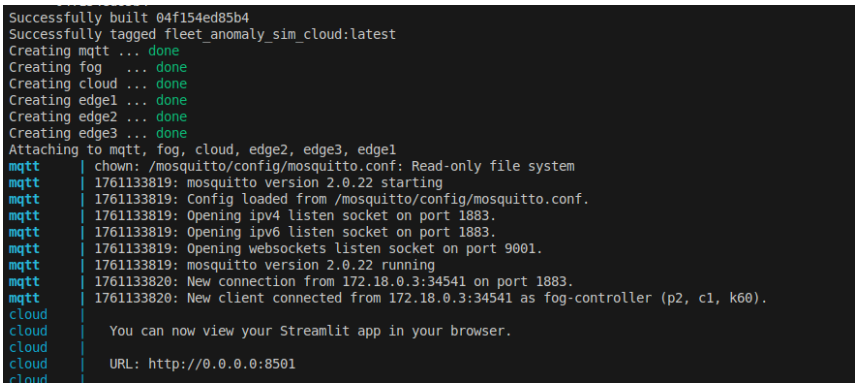


Figure 3: Creating and running docker containers for edge, fog and cloud layers.

6.2 Unsupervised Learning for Anomaly Detection

- **Hierarchical Clustering:** The framework applies agglomerative hierarchical clustering to group machines by pairwise similarity. Each machine begins as its own cluster, and merges occur iteratively based on linkage criteria until a full dendrogram is formed. A **correlation threshold** is then used to cut the dendrogram into meaningful clusters, avoiding arbitrary distance cutoffs and ensuring structure is guided by the data itself.

- **Anomaly Identification:** The assumption is that most machines in the fleet operate under healthy conditions. Hence, large clusters representing the majority of machines are considered *normal*, while smaller clusters — containing fewer machines — are flagged as *anomalous*. This reflects the principle that faults are rare deviations from dominant behavior.

- **Anomaly Scoring:** Each machine is assigned a fraction-based anomaly score that quantifies how isolated it is from the rest of the fleet:

$$\text{Anomaly Score}_i = 1 - \frac{\text{Size of Cluster}_i}{\text{Total Number of Machines}}$$

  Scores near zero indicate strong normality (membership in large clusters), while higher scores suggest isolation or abnormal patterns. A threshold — typically around $\frac{2}{3}$ — is used to classify machines as normal or anomalous:

$$\text{Machine}_i = \begin{cases} \text{Normal}, & \text{if Anomaly Score}_i < \frac{2}{3}, \\ \text{Anomalous}, & \text{otherwise.} \end{cases}$$

- **Visualization and Interpretability:** To ensure transparency and explainability, several plots are generated to visualize clustering and anomalies:

  - **Normalized Signal Plots:** Compare individual machine signals on a common scale.
  - **Dissimilarity Matrix:** Displays pairwise distances and highlights clusters and outliers.
  - **Dendrogram:** Shows hierarchical relationships and the correlation cut threshold.
  - **Anomaly Score Distribution:** Provides a fleet-level overview of machine health.

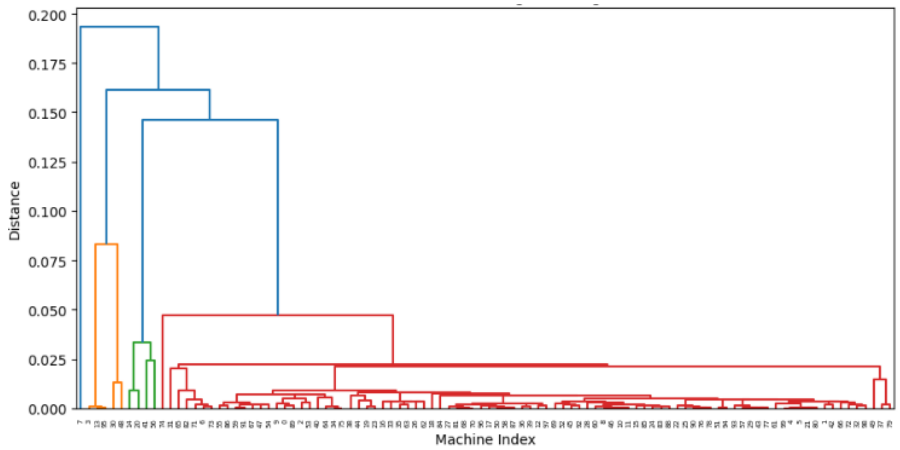


Figure 4: Dendogram of a fleet of machines, computed by an edge node.

- **Benchmark Method:** For performance evaluation, the framework is compared against a simpler statistical rule-based approach. In the benchmark, a machine is marked anomalous if its measurement deviates by more than a few standard deviations from the fleet mean:

$$|x_i - \mu| > k\sigma, \quad \text{where } k \text{ is a sensitivity parameter.}$$

  This comparison highlights the improved adaptability and interpretability of the clustering-based method over conventional threshold-based rules.

- **Summary:** The hierarchical clustering framework provides a principled, transparent, and data-driven approach to anomaly detection. Its emphasis on visualization and interpretability makes it well-suited for industrial monitoring and predictive maintenance applications.

7 Results and Discussions

7.1 Accuracy

The performance of the proposed unsupervised anomaly detection model shows a significant improvement as the fleet size increases. When evaluated on a smaller fleet of 50 machines, the model achieved an accuracy of 93%. However, as the fleet size expanded to 300 machines, the accuracy rose to 97%. This improvement can be attributed to the greater diversity and volume of sensor data available in larger fleets, which allows the clustering and similarity-based algorithms to better learn the representation of normal machine behavior. The increased data variety enhances the model's ability to distinguish between true anomalies and natural operational variations, leading to more stable and reliable fault detection across the system.

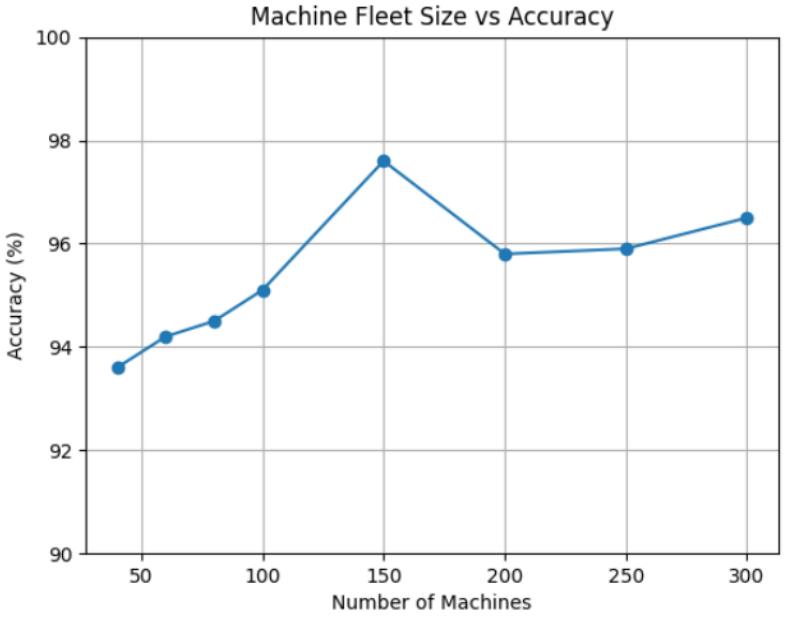


Figure 5: Machine fleet size vs. accuracy.

7.2 End-to-End Latency

End-to-End Latency: The system's transmission performance was evaluated based on message size and network capacity.

- **Message size for 100 machines:** 180 KB
- **Network transmission rate:** 100 Mbps

The propagation delay is considered negligible compared to the transmission delay.

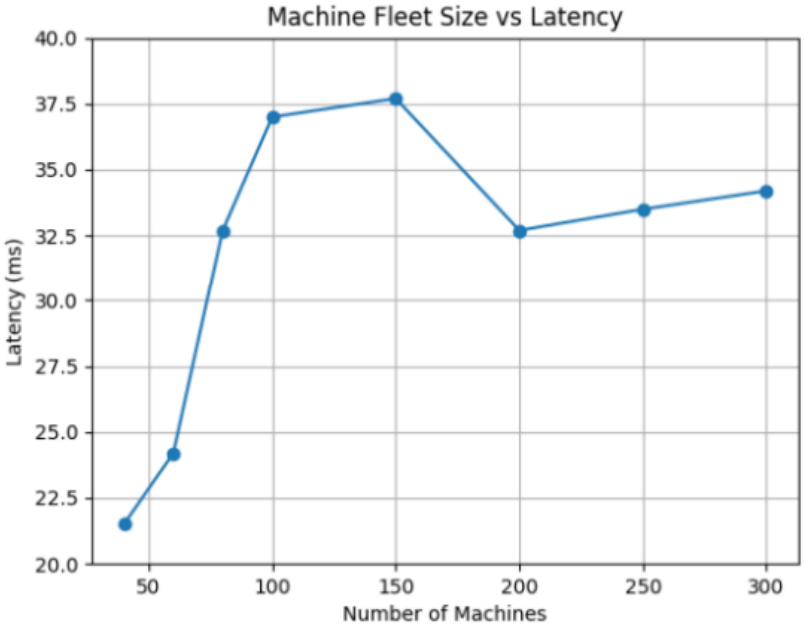


Figure 6: Fleet size vs. end-to-end latency.

7.3 Processing power

When the number of machines grew from 50 to 300, the CPU utilization of the edge node rose from 10% to 40%. This rise is expected, as a larger fleet generates more sensor data that must be processed locally for clustering and anomaly detection. The utilization remains within acceptable limits for real-time edge operations, demonstrating that the proposed framework maintains reliable performance.

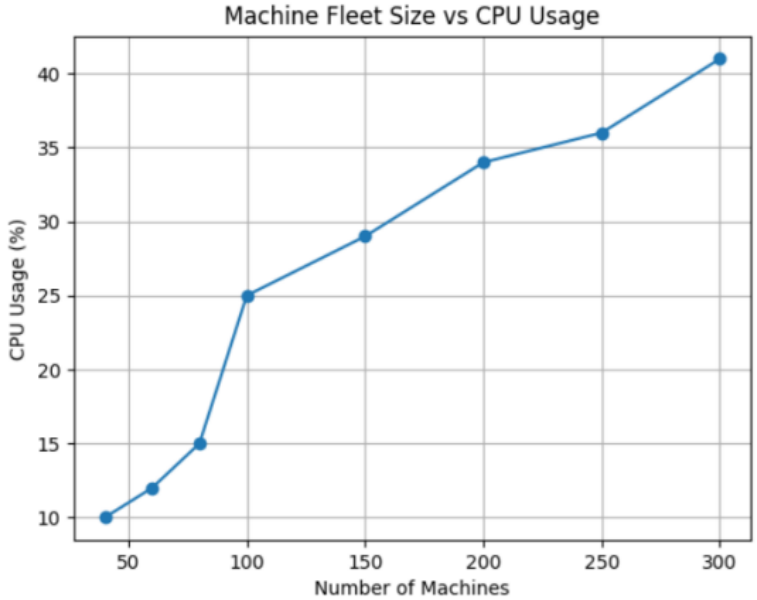


Figure 7: Fleet size vs. CPU utilization.

The RAM utilization followed a similar upward trend, increasing from 10 MB to 140 MB as the fleet size expanded from 50 to 300 machines. A sudden spike in memory usage was observed around 200 machines, which corresponds to the point where the system begins handling multiple large data buffers simultaneously. This behavior can be attributed to the higher data throughput and the growing number of active clustering computations required for real-time anomaly detection.

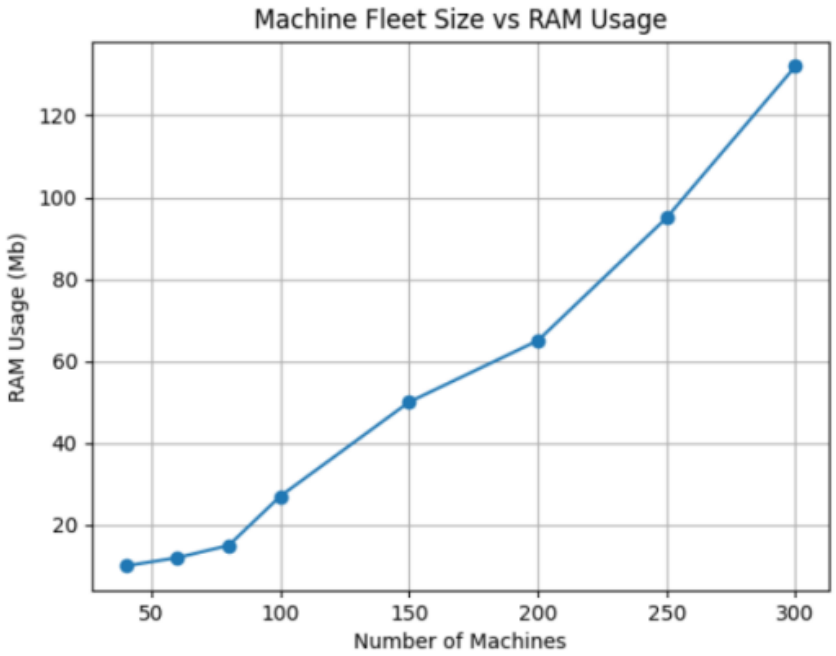
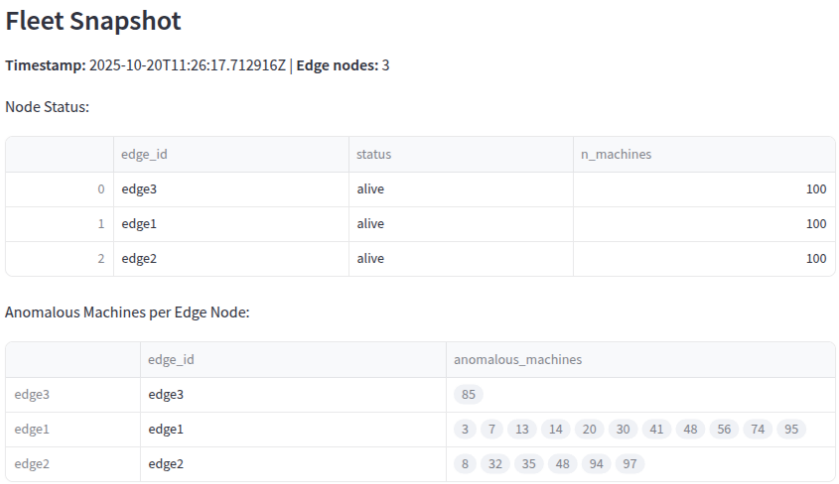


Figure 8: Fleet size vs. RAM utilization.

7.4 Fleet Snapshot

What is shown: A compact table listing each edge node, the time of the latest message, and the machine indices flagged as anomalous by local models.
Why it matters: Operators can immediately identify which nodes require attention and start investigation workflows. It is the first level of triage that prioritizes limited maintenance resources.
Operational insight: This view confirms that the simulated edges publish regularly and that the central UI collects and displays metadata stores reliably. It also demonstrates the value of showing flagged indices rather than raw telemetry for quick triage.



Fleet Snapshot

Timestamp: 2025-10-20T11:26:17.712916Z | **Edge nodes:** 3

Node Status:

	edge_id	status	n_machines
0	edge3	alive	100
1	edge1	alive	100
2	edge2	alive	100

Anomalous Machines per Edge Node:

	edge_id	anomalous_machines
edge3	edge3	85
edge1	edge1	3 7 13 14 20 30 41 48 56 74 95
edge2	edge2	8 32 35 48 94 97

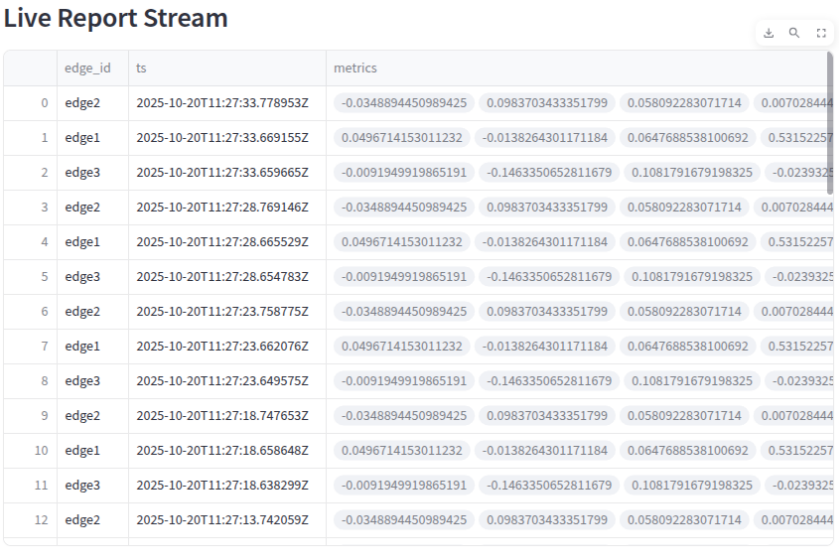
Figure 9: Fleet Snapshot: per-edge node status, timestamps and anomalous machine indices.

7.5 Live report stream

What is shown: The raw stream of metric vectors (normalized), each associated with a timestamp and an edge identifier.

Why it matters: During development and debugging this stream verifies message order, latency, and parsing. For operations, it can indicate transient anomalies correlated to specific events like startup spikes.

Operational insight: A stable stream with little jitter indicates the messaging and edge simulators are performing reliably. Missing rows or large gaps would point to networking issues or crashed publishers.



Live Report Stream

	edge_id	ts	metrics
0	edge2	2025-10-20T11:27:33.778953Z	-0.0348894450989425 0.0983703433351799 0.058092283071714 0.007028444
1	edge1	2025-10-20T11:27:33.669155Z	0.0496714153011232 -0.0138264301171184 0.0647688538100692 0.53152257
2	edge3	2025-10-20T11:27:33.659665Z	-0.0091949919865191 -0.1463350652811679 0.1081791679198325 -0.023932
3	edge2	2025-10-20T11:27:28.769146Z	-0.0348894450989425 0.0983703433351799 0.058092283071714 0.007028444
4	edge1	2025-10-20T11:27:28.665529Z	0.0496714153011232 -0.0138264301171184 0.0647688538100692 0.53152257
5	edge3	2025-10-20T11:27:28.654783Z	-0.0091949919865191 -0.1463350652811679 0.1081791679198325 -0.023932
6	edge2	2025-10-20T11:27:23.758775Z	-0.0348894450989425 0.0983703433351799 0.058092283071714 0.007028444
7	edge1	2025-10-20T11:27:23.662076Z	0.0496714153011232 -0.0138264301171184 0.0647688538100692 0.53152257
8	edge3	2025-10-20T11:27:23.649575Z	-0.0091949919865191 -0.1463350652811679 0.1081791679198325 -0.023932
9	edge2	2025-10-20T11:27:18.747653Z	-0.0348894450989425 0.0983703433351799 0.058092283071714 0.007028444
10	edge1	2025-10-20T11:27:18.658648Z	0.0496714153011232 -0.0138264301171184 0.0647688538100692 0.53152257
11	edge3	2025-10-20T11:27:18.638299Z	-0.0091949919865191 -0.1463350652811679 0.1081791679198325 -0.023932
12	edge2	2025-10-20T11:27:13.742059Z	-0.0348894450989425 0.0983703433351799 0.058092283071714 0.007028444

Figure 10: Live Report Stream: timestamped metric vectors from edge nodes.

8 Conclusion

This research presents an edge-based anomaly detection framework for fleet-based condition monitoring of machines. The proposed edge-fog-cloud architecture enables real-time fault detection at the network edge while reducing latency, bandwidth consumption, and data privacy risks. By leveraging unsupervised anomaly detection through hierarchical clustering, the framework successfully identifies machine anomalies without requiring historical labeled failure data. Experimental simulations demonstrate that the distributed architecture achieves low-latency fault detection, scalable fleet management, and effective anomaly visualization suitable for industrial IoT environments with limited connectivity.

9 Future Work

- Integration of deep learning or hybrid models to improve anomaly detection accuracy.
- Optimization of edge node resources (CPU, RAM, energy) for handling larger fleets efficiently.
- Real-world deployment and testing on actual industrial machines to validate performance.
- Incorporation of security measures, such as encryption and authentication, for safe data communication.

References

[1] K. Hendrickx *et al.*, "A general anomaly detection framework for fleet-based condition monitoring of machines," *Mech. Syst. Signal Process.*, vol. 139, p. 106585, 2020.

[2] H. T. Truong *et al.*, "Light-weight federated learning-based anomaly detection for time-series data in industrial control systems," *Comput. Ind.*, vol. 140, p. 103692, 2022.

[3] M. L. D. Wong, L. B. Jack, and A. K. Nandi, "Modified self-organising map for automated novelty detection applied to vibration signal monitoring," *Mech. Syst. Signal Process.*, vol. 20, no. 3, pp. 593–610, 2006.

[4] C. Derse *et al.*, "An anomaly detection study on automotive sensor data time series for vehicle applications," in *Proc. 16th Int. Conf. Ecol. Vehicles Renewable Energies (EVER)*, Monte-Carlo, Monaco, 2021, pp. 1–5.

[5] S. T. S., S. M. Rajagopal, and S. Bhaskaran, "Comparative analysis of machine learning algorithms for anomaly detection," in *Proc. IEEE 9th Int. Conf. Convergence Technol. (I2CT)*, Pune, India, 2024, pp. 1–5.

[6] M. Carratù *et al.*, "A novel methodology for unsupervised anomaly detection in industrial electrical systems," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–12, 2023, Art. no. 3532812.

[7] J. Zou, X. Jin, L. Zhang, Y. Wang, and B. Li, "A case study of anomaly detection in industrial environments," in *Proc. IEEE Int. Conf. Comput. Sci. Eng. (CSE) and IEEE Int. Conf. Embedded Ubiquitous Comput. (EUC)*, New York, NY, USA, 2019, pp. 294–298.

[8] V. Radhakrishnan, N. Kabilan, V. Ravi, and V. Sowmya, "Unsupervised representation learning approach for intrusion detection in the industrial Internet of Things network environment," in *Analytics Modeling in Reliability and Machine Learning and Its Applications*, H. Pham, Ed. Cham, Switzerland: Springer, 2025, ch. 3.

[9] K. G, S. Sivakumar, and S. Manickam, "Optimized hybrid approach for anomaly detection of DDoS and network attacks in IoMT systems using autoencoders and TabNet," in *Proc. 4th Int. Conf. Mobile Netw. Wireless Commun. (ICMNWC)*, Tumkuru, India, 2024, pp. 1–7.

[10] D. Abhiram, K. D. Reddy, P. K. Reddy, and S. Rajagopal, "IoT environmental monitoring with anomaly detection," in *Proc. 3rd Int. Conf. Intell. Data Commun. Technol. Internet Things (IDCIoT)*, Bengaluru, India, 2025, pp. 699–704.